

Implementation of an Intelligent Child Hazard Monitoring System

Date de soutenance : **26 août 2026**

Stage 2A 2025/2026

Stage du 14 avril – 14 août 2026

Nisrine Khnifass,
Année Universitaire 2025/2026
Spécialité Informatique

Tuteur entreprise : Muhammad Khan
Tuteur ENSICAEN : Karim-Eric Ziad-Forest



Remerciements

Je remercie le professeur Muhammad Khan de m'avoir accueillie au sein du Visual Analytics for Knowledge Laboratory (VIS2KNOW Lab) de Sungkyunkwan University et de m'avoir confié ce sujet de recherche. Je remercie également Shehzad Ali, chercheur au VIS2KNOW Lab, pour sa disponibilité quotidienne et les échanges scientifiques qui ont guidé les différentes étapes expérimentales de ce travail. Enfin, je remercie Karim-Eric Ziad-Forest pour le suivi pédagogique assuré au nom de l'ENSICAEN, ainsi que l'ensemble des membres du laboratoire pour leur accueil.

Table des matières

Remerciements	I
Introduction	1
1 Présentation de la structure d'accueil	1
2 Sujet, contexte et problématique	1
2.1 Contexte général	1
2.2 Étude de l'existant et positionnement	1
2.3 Problématique et objectifs mesurables	2
3 Travail demandé	2
3.1 Objectif global et architecture cible	2
3.2 Modélisation UML et patrons de conception	3
4 Travail personnel effectué	3
4.1 Vue d'ensemble des contributions	3
4.2 Module 1 – Détection de l'enfant	4
4.3 Module 2 – Reconnaissance de l'activité	4
4.4 Module 3 – Extraction du contexte	6
4.5 Modules 4 et 5 – Décision du risque et notification	9
4.6 Tests logiciels	10
4.7 Validation de bout en bout et limites	10
5 Méthode de gestion de projet	11
5.1 Organisation adoptée	11
5.2 Rétrospective : et si c'était à refaire ?	11
6 Bilan et conclusion	12
6.1 Bilan : ce que ce stage m'a apporté	12
6.2 Conclusion	12
Annexes	14
A Paramètres principaux des expériences	15
B Conventions d'annotation manuelle (CVAT, benchmark objets)	15
C Résultats complémentaires du benchmark d'objets	15

Introduction

Les accidents domestiques impliquant de jeunes enfants peuvent survenir en quelques secondes, alors qu'une surveillance humaine continue n'est pas toujours possible. Dans ce contexte, l'analyse automatique de vidéos peut apporter une aide en détectant l'enfant, son activité et les objets présents dans son environnement avant d'estimer le niveau de danger.

Ce stage, réalisé du 14 avril au 14 août 2026 au VIS2KNOW Lab de Sungkyunkwan University, avait pour objectif de construire et d'évaluer les principales briques d'une chaîne modulaire de surveillance. La démarche a consisté à comparer plusieurs modèles, à contrôler la validité des protocoles expérimentaux, puis à définir des interfaces explicites entre perception et raisonnement. Le rapport présente successivement la structure d'accueil, le sujet et l'existant, le travail demandé, les contributions expérimentales, la gestion du projet, puis le bilan et les limites du prototype.

1 Présentation de la structure d'accueil

Le stage s'est déroulé en présentiel, du 14 avril au 14 août 2026, à la Sungkyunkwan University (SKKU), fondée en 1398 [13], au sein du *Department of Applied Artificial Intelligence*. Le *Visual Analytics for Knowledge Laboratory* (VIS2KNOW Lab) y mène des travaux en intelligence artificielle, surveillance intelligente et analyse visuelle, sous la direction du professeur Muhammad Khan, avec un accompagnement scientifique et technique quotidien assuré par Shehzad Ali, chercheur au laboratoire.

L'encadrement reposait sur trois niveaux complémentaires : le professeur Muhammad Khan assurait la direction scientifique du sujet, Shehzad Ali apportait un suivi technique régulier au laboratoire et Karim-Eric Ziad-Forest assurait le suivi pédagogique pour l'ENSICAEN. Le travail expérimental a été réalisé principalement en autonomie, avec présentation des choix et des résultats intermédiaires aux encadrants.

Le sujet s'inscrit dans la continuité des travaux du laboratoire sur la reconnaissance d'activités dangereuses des enfants en intérieur. Au début du stage, le laboratoire disposait d'un jeu de données de détection de l'enfant, du dataset vidéo RHAD (section 2.2), de dépôts de modèles pré-entraînés et d'une architecture conceptuelle. En revanche, le prototype intégré reliant détection, reconnaissance d'activité, contexte et décision de danger n'existait pas. Ma contribution a consisté à adapter et comparer les modèles de perception, à développer les briques de contexte et de décision, puis à définir leurs interfaces afin d'obtenir une chaîne testable.

2 Sujet, contexte et problématique

2.1 Contexte général

Chez le jeune enfant, l'accident domestique (chute, contact électrique, manipulation d'un objet tranchant, chute d'un objet) se caractérise par une cinétique extrêmement rapide, qui exige une réponse humaine en quelques secondes. La vigilance humaine continue étant structurellement impossible, et les caméras de surveillance intérieures restant conditionnées à une observation humaine active, l'objectif de ce projet est d'intégrer une couche d'analyse automatisée du flux vidéo qui condense un signal visuel continu en une information sémantique concise et exploitable en temps réel.

2.2 Étude de l'existant et positionnement

Le point de départ du laboratoire est le *Real-World Hazardous Activities Dataset* (RHAD) et le modèle HAZARDNET [1], qui combine extraction spatiale légère et raisonnement temporel pour classifier des activités intérieures (normales, dommageables, dangereuses). Ce travail démontre l'intérêt d'une représentation spatio-temporelle mais se limite à la classification de l'activité, alors qu'une décision de sécurité fiable nécessite aussi des informations sur l'environnement (objets présents, distance à l'enfant, durée de l'interaction). C'est le principal manque que ce stage cherche à combler.

Trois familles de modèles ont guidé les choix techniques : les détecteurs d'objets YOLO et les détecteurs *Transformer* de type RT-DETR [5] pour la localisation ; les modèles de reconnaissance vidéo VIDEOMAE [2], UNIFORMERV2 [3] et VIDEOMAMBA [4], choisis car ils représentent trois approches architecturales distinctes (Transformer masqué, Transformer hybride, modèle d'état) ; enfin les détecteurs *open-vocabulary* YOLO-World [7], Grounding DINO [8], OWLv2 [9] et YOLOE [10], capables d'identifier des objets à partir de requêtes textuelles sans réentraînement dédié.

Le positionnement méthodologique du stage, par rapport à HazardNet, est de conserver les résultats intermédiaires de chaque module plutôt que de produire directement une décision de bout en bout opaque. Cette traçabilité permet d'expliquer *pourquoi* une alerte est générée et de distinguer une erreur de perception

(activité ou objet incorrect) d'une erreur de raisonnement (mauvaise interprétation d'informations pourtant correctes).

2.3 Problématique et objectifs mesurables

Problématique.

Comment construire et évaluer une chaîne modulaire capable de détecter un enfant, de reconnaître son activité, d'extraire les éléments contextuels pertinents, puis d'estimer le risque afin de générer une alerte exploitable et explicable ?

Module	Rôle	Objectif mesurable
1	Détection de l'enfant	Maximiser le mAP@50-95 et le F1, sous contrainte de taille du modèle
2	Reconnaissance d'activité	Maximiser le Macro F1 sur un split <i>sans fuite</i> , rappel des activités anormales proche de 100 %
3	Extraction du contexte	JSON par vidéo : objets, distance normalisée à l'enfant, persistance temporelle
4-5	Décision & notification	Décision structurée (JSON valide) et notification journalisée

Tableau 1: Objectifs mesurables par module, tels que fixés en début de stage.

Une attention particulière est portée, tout au long de la recherche, à l'analyse critique des erreurs plutôt qu'à la seule optimisation d'un score global.

3 Travail demandé

3.1 Objectif global et architecture cible

Le travail demandé porte sur l'entraînement et l'évaluation comparative des modèles de vision constituant les modules 1 à 4, la conception du format d'échange entre perception et raisonnement, et l'intégration de l'ensemble en un prototype testable. L'architecture cible est organisée en cinq modules, alimentant tous un document JSON commun (figure 1), qui constitue l'interface explicite entre perception et raisonnement – le choix de conception central du système : il rend chaque étape testable indépendamment et permet, en cas d'alerte incorrecte, de localiser le module responsable.

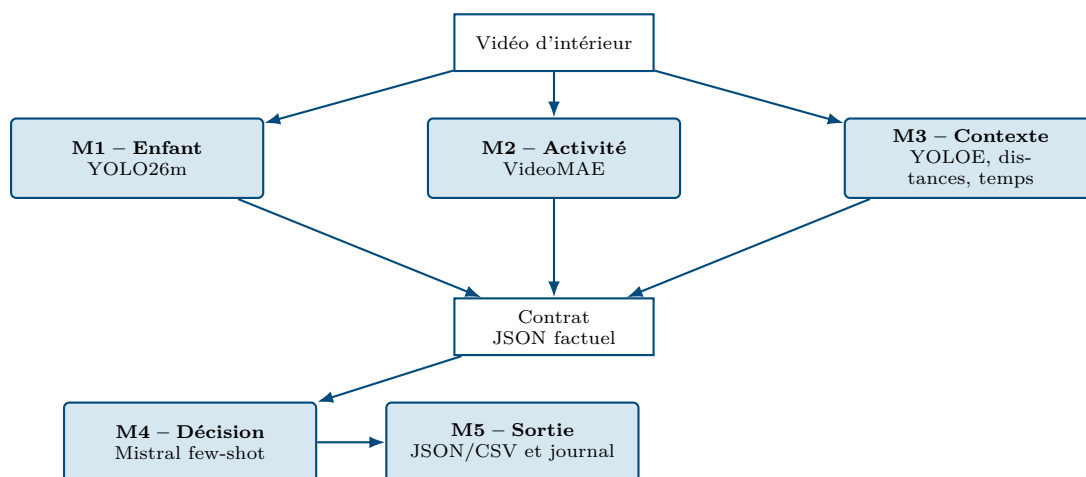


Figure 1: Architecture modulaire du prototype. Les modules de perception produisent des faits traçables, fusionnés dans un contrat JSON avant la décision de danger et sa journalisation.

Cette vue de composants met en œuvre deux principes d'architecture : la séparation des responsabilités (perception, interprétation, sortie) et un contrat de données explicite. Le code étant organisé principalement en notebooks et fonctions de pipeline plutôt qu'en une hiérarchie de classes complète, les patrons de conception présentés en section 3.2 décrivent la logique effective de conception – interfaces stables, composants interchangeables – plutôt qu'une implémentation orientée objet canonique au sens strict.

3.2 Modélisation UML et patrons de conception

Le diagramme de cas d'utilisation ci-dessous décrit l'usage visé du système complet : le prototype actuel réalisé pendant le stage implémente le cœur du traitement (caméra vers décision, section 4.5), tandis que les cas d'utilisation côté aidant et opérateur décrivent les fonctions destinées à une version déployée (figure 2).

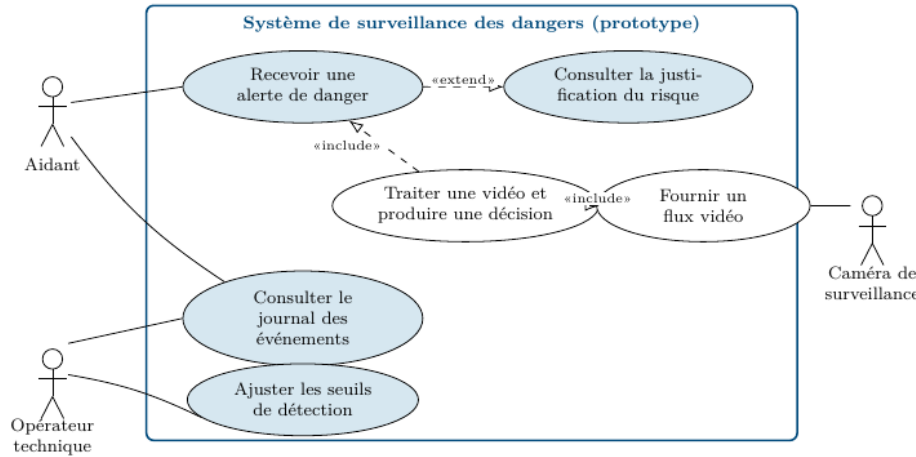


Figure 2: Diagramme de cas d'utilisation du système cible.

Trois patrons de conception structurent l'architecture effective du système, résumés dans le tableau 2. Le code étant organisé en notebooks et fonctions plutôt qu'en une hiérarchie de classes complète, ces patrons décrivent la logique de conception – interfaces stables, composants interchangeables – plutôt qu'une implémentation orientée objet canonique au sens strict.

Patron	Où il s'applique	Justification
Pipeline (pipe-and-filter)	Architecture globale (M1 à M5)	Chaque module transforme et enrichit un flux de données jusqu'à la décision finale ; permet de tester ou remplacer un maillon sans toucher aux autres.
Strategy	Détecteur d'objets (M3), moteur de décision (M4)	Cinq détecteurs et quatre moteurs de décision partagent la même interface d'entrée/sortie, ce qui a permis de les comparer et de les substituer sans modifier le reste du pipeline.
Facade	Contrat JSON entre perception (M1-M3) et raisonnement (M4)	Le JSON expose une interface simple et stable au Module 4, qui masque la complexité et l'hétérogénéité des trois modules de perception.

Tableau 2: Patrons de conception structurant l'architecture du prototype.

4 Travail personnel effectué

4.1 Vue d'ensemble des contributions

- Comparaison de neuf configurations de détection de l'enfant (famille YOLO + RT-DETR) → sélection de YOLO26m.
- Construction d'un protocole d'évaluation commun pour trois modèles vidéo, après identification et correction d'une fuite de données.
- Sélection de VideoMAE et étude quantitative d'un seuil de décision orienté sécurité (abandonné après analyse – section 4.3).
- Enrichissement synthétique de la classe rare `objects_falling` et *fine-tuning* (expérience négative documentée).
- Développement d'un pipeline de contexte : détection ouverte d'objets (5 modèles comparés sur un benchmark annoté de 96 images), proximité enfant–objet, stabilisation temporelle.
- Comparaison de 6 configurations de décision du risque (règles, 4 LLM, adaptation QLoRA) sur un jeu de test annoté manuellement.

- Définition des interfaces, intégration progressive des modules et validation logicielle de leurs sorties ; préparation d'un test de bout en bout non encore archivé.

4.2 Module 1 – Détection de l'enfant

Le jeu de données **ChildDetection** fourni par le laboratoire contient **2 257 images annotées** pour la classe unique **child**, au format YOLO : 2 192 images d'entraînement, 50 de validation et 15 de test. La taille réduite des ensembles de validation et de test, imposée par les données disponibles, limite la précision statistique des métriques et doit être prise en compte dans leur interprétation. Tous les entraînements utilisent des images 640×640 sur GPU Tesla T4. La comparaison est menée en trois étapes : variantes YOLO *nano* pour un tri rapide, candidats *medium* [6], puis comparaison à l'architecture Transformer RT-DETR L [5].

Modèle	mAP@50	mAP@50-95	Précision	Rappel
YOLOv8n	0.972	0.857	0.966	0.918
YOLOv9t	0.978	0.858	0.985	0.890
YOLOv10n	0.966	0.848	0.968	0.959
YOLO11n	0.970	0.848	0.943	0.932
YOLO12n	0.959	0.847	0.921	0.959
YOLO26n	0.959	0.884	0.947	0.890

Tableau 3: Comparaison des variantes YOLO nano (200 époques, batch 16).

La sélection ne peut pas reposer sur une seule métrique. Le rappel mesure la capacité à ne pas manquer l'enfant, tandis que la mAP@50-95 pénalise les boîtes mal localisées qui dégraderaient ensuite les distances enfant-objet. YOLO26n est donc retenu pour l'étape suivante en raison de sa meilleure mAP@50-95, tout en conservant comme exigence de contrôler son rappel lors de l'intégration.

Modèle	Époques	mAP@50	mAP@50-95	F1	Taille
YOLO11m	175	0.967	0.894	0.934	40.5 MB
YOLO26m	200	0.989	0.935	0.957	44.0 MB
RT-DETR L	100	0.976	0.915	0.954	66.2 MB

Tableau 4: Comparaison initiale des détecteurs retenus (taille medium et Transformer).

Cette première comparaison conduit à retenir la famille YOLO26m : RT-DETR L est environ 50 % plus volumineux sans gain de qualité correspondant.

YOLO26m améliore simultanément précision, rappel et qualité de localisation ; il est donc intégré au pipeline final.

Analyse des erreurs. Les essais visuels réalisés lors de l'intégration du Module 3 montrent que la détection se dégrade lorsque l'enfant est partiellement masqué ou que son apparence est altérée par le mouvement. Cette observation motive l'usage d'un seuil de détection de secours plus permissif lors de l'extraction du contexte (section 4.4) plutôt qu'un seuil fixe unique.

4.3 Module 2 – Reconnaissance de l'activité

Correction d'un biais méthodologique. La version de RHAD utilisée contient **3 248 vidéos** en douze classes. Un premier découpage aléatoire *au niveau des clips* a produit des performances anormalement élevées : plusieurs clips dérivés d'une même vidéo source étaient répartis dans des ensembles différents – une fuite de données classique dans les jeux de vidéos courtes.

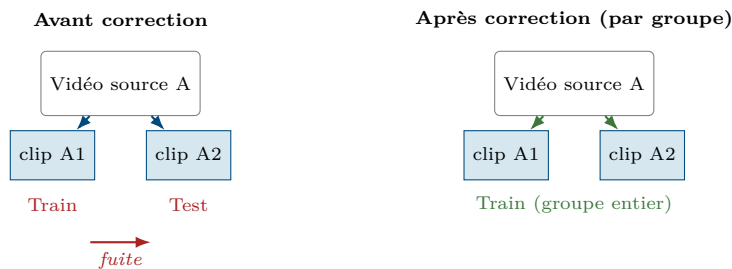


Figure 3: Correction du protocole : les clips d’une même vidéo source sont désormais affectés en bloc à un seul ensemble.

La figure 3 illustre la correction appliquée au découpage.

L’audit initial du jeu de données a mis en évidence plusieurs menaces à la validité : des clips identiques ou presque identiques, une forte redondance des mouvements au sein de certaines classes, ainsi que du texte incrusté, des filigranes, du flou et des artefacts de compression. Ces éléments peuvent devenir des raccourcis visuels appris par le réseau sans relation causale avec l’activité. Le cas le plus critique était le découpage d’une même vidéo source en cinq clips susceptibles d’être distribués entre entraînement et test, conduisant à une fuite de données.

Pour réduire ce biais, les cinq clips dérivés d’une même séquence source sont désormais maintenus dans un même groupe. Avec une graine fixée à 42, le protocole corrigé donne 2578 vidéos d’entraînement, 335 de validation et 335 de test, sans groupe partagé. Les trois modèles candidats ont été réentraînés sur ce découpage commun. Cette correction supprime la fuite identifiée, mais ne suffit pas à éliminer toute redondance sémantique entre des scènes différentes ; l’analyse par classe reste donc indispensable.

Modèle	Top-1	Macro F1	F1 pondéré	Rappel anormal	Spécificité normale
VideoMAE	91.34%	89.68%	91.36%	100.00%	71.43%
UniFormerV2	89.25%	88.15%	89.14%	100.00%	57.14%
VideoMamba Middle	85.08%	77.92%	84.28%	99.33%	71.43%

Tableau 5: Comparaison standardisée sur validation (découpage corrigé).

VideoMAE est retenu : meilleur Macro F1, meilleur rappel des événements anormaux, spécificité comparable au meilleur concurrent. Sur le test indépendant (335 vidéos), il atteint 88.657 % d’accuracy Top-1, 100 % en Top-5, un Macro F1 de 0.8151 et un F1 pondéré de 0.8818. Le Macro F1, qui accorde le même poids à chaque classe, est inférieur au F1 pondéré : les classes minoritaires ou difficiles restent donc moins bien reconnues que ne le suggère l’accuracy globale. Ramenées à la décision binaire utile au système (anormal / normal), les prédictions conduisent à la matrice de la figure 4 : les 300 événements anormaux sont reconnus, avec deux fausses alertes sur 35 vidéos normales.

		Prédiction	
		Sans alerte	Alerte
Vérité terrain	Normal	33	2
	Anormal	0	300

Figure 4: Matrice de confusion binaire de VideoMAE sur les 335 vidéos de test : 33 vrais négatifs, 2 faux positifs, aucun faux négatif et 300 vrais positifs.

Étude du seuil de décision AT . Une alerte d’activité est formalisée par

$$\text{Alerte}_{\text{activité}} = \mathbb{1}[\hat{C} \neq C_{\text{normal}} \wedge P_{\text{anormal}} \geq AT], \quad P_{\text{anormal}} = 1 - P(C_{\text{normal}}). \quad (1)$$

Le seuil a été recherché uniquement sur les 335 vidéos de validation parmi 671 valeurs candidates, avec la contrainte prioritaire de conserver un rappel anormal de 100 %. Cette procédure conduit à $AT = 0.354459$, ensuite gelé avant l’évaluation sur le test indépendant.

Mesure	Sans filtrage	Avec $AT = 0.354459$
Événements anormaux détectés	300/300	300/300
Faux négatifs	0	0
Fausse alerte sur vidéos normales	2/35	2/35
Spécificité normale	94.286 %	94.286 %

Tableau 6: Résultat binaire de VideoMAE sur le test avec $AT = 0.354459$.

Le seuil final ne modifie donc aucune décision sur le test : il ne doit pas être présenté comme une amélioration des performances. Cette expérience montre au contraire qu'un seuil softmax ne fournit ici aucun gain opérationnel démontré. Dans la chaîne finale, la classe et la confiance sont transmises au Module 4 ; la confiance sert d'indicateur d'incertitude, pas de preuve suffisante d'absence de danger.

Enrichissement de la classe `objects_falling`. L'analyse par classe révèle que les cinq vidéos de test `objects_falling` sont systématiquement confondues avec `falling` (confiance 0.81–0.86). Onze vidéos synthétiques ont été générées (Hailuo AI), déclinées en quatre variantes chacune (retournement, luminosité, dégradation qualité caméra, accélération $\times 2$), portant la classe de 60 à 115 vidéos d'entraînement (dataset total : 3 248 $\rightarrow$ 3 303). Après *fine-tuning* (5 époques, 162 tenseurs modifiés), les prédictions sur les 335 vidéos de test restent quasiment inchangées (88.66 % d'exactitude, mêmes 5 erreurs) : une expérience négative, retenue comme telle. L'explication la plus probable est que, dans la majorité des scènes générées, l'enfant tombe après avoir été touché par l'objet. Le modèle apprend surtout le mouvement de la chute de l'enfant, déjà associé à `falling`, plutôt que celui de l'objet qui la précède.

4.4 Module 3 – Extraction du contexte

Le Module 3 complète l'activité reconnue par des informations sur l'environnement immédiat de l'enfant, ce qui manquait à HazardNet (section 2.2) et permet, en particulier, de distinguer une chute *de* l'enfant d'une chute d'objet *près de* l'enfant.

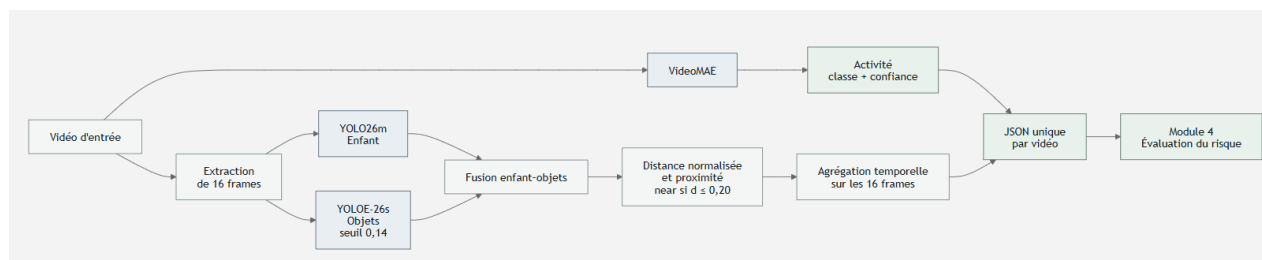


Figure 5: Pipeline effectivement implémenté pour l'extraction du contexte : VideoMAE traite la vidéo, tandis que 16 frames alimentent les détecteurs enfant et objets avant fusion géométrique et agrégation temporelle.

La figure 5 synthétise les traitements effectivement implémentés et leurs échanges de données.

Détection de l'enfant dans le lot d'intégration (192 images, 12 vidéos). Une première passe (seuil 0.25) détecte l'enfant dans 79.17 % des images ; une seconde passe, appliquée uniquement aux images sans détection (seuil de secours 0.10), porte ce taux à 83.33 % (+8 détections). Ce résultat reste hétérogène selon l'activité :

Activité(s)	Taux
child_running, climbing, normal, scribbling, sharp_objects, electrical_appliances	100 %
collision, falling	93.75 %
objects_falling	68.75 %
Jumping, pulling_curtains	62.50 %
head_stuck	18.75 %
Moyenne globale	83.33 %

Tableau 7: Taux de détection de l'enfant par classe d'activité (192 images, 12 vidéos).

La classe `head_stuck` est la principale faiblesse identifiée : l'enfant y est souvent partiellement masqué ou dans une posture inhabituelle. Cette limite est jugée critique car, sans localisation de l'enfant, aucune relation géométrique enfant-objet n'est calculable.

Sélection du détecteur d'objets contextuels. Cinq détecteurs *open-vocabulary* ont été comparés sur 96 images extraites de 24 vidéos, soit deux vidéos par classe d'activité et quatre images par vidéo. Les images ont été réparties entre un ensemble de calibration et un ensemble de test de 48 images chacun, sans vidéo commune, afin d'éviter toute fuite de données.

Les annotations, réalisées manuellement avec CVAT, comprennent 442 boîtes réparties entre 44 catégories : l'enfant et 43 objets contextuels. L'enfant étant détecté séparément par YOLO26m, seuls les 43 objets ont été évalués. Pour garantir une comparaison équitable, tous les modèles ont été testés sur les mêmes images, avec les mêmes annotations et les mêmes requêtes.

Le seuil de confiance de chaque modèle a été choisi sur l'ensemble de calibration en maximisant le F1, puis conservé pour le test. Une détection était considérée comme correcte lorsque sa catégorie correspondait à l'annotation et que l'IoU était supérieure ou égale à 0.50. Les performances ont ensuite été comparées à partir du score F1, de la mAP@0.50 et du nombre d'images traitées par seconde.

Modèle	F1	mAP@0.50	Débit	Analyse
OWLv2	0.361	0.458	~1.2 img/s	Meilleure qualité, trop lent
YOLOE-26s	0.277	0.430	~50 img/s	Meilleur compromis
YOLO-World	0.272	0.345	Rapide	mAP plus faible
OmDet-Turbo	0.263	0.425	Intermédiaire	Qualité proche, plus lent
Grounding DINO	0.065	0.081	Lent	Performances insuffisantes

Tableau 8: Comparaison des cinq détecteurs open-vocabulary sur le test (48 images).

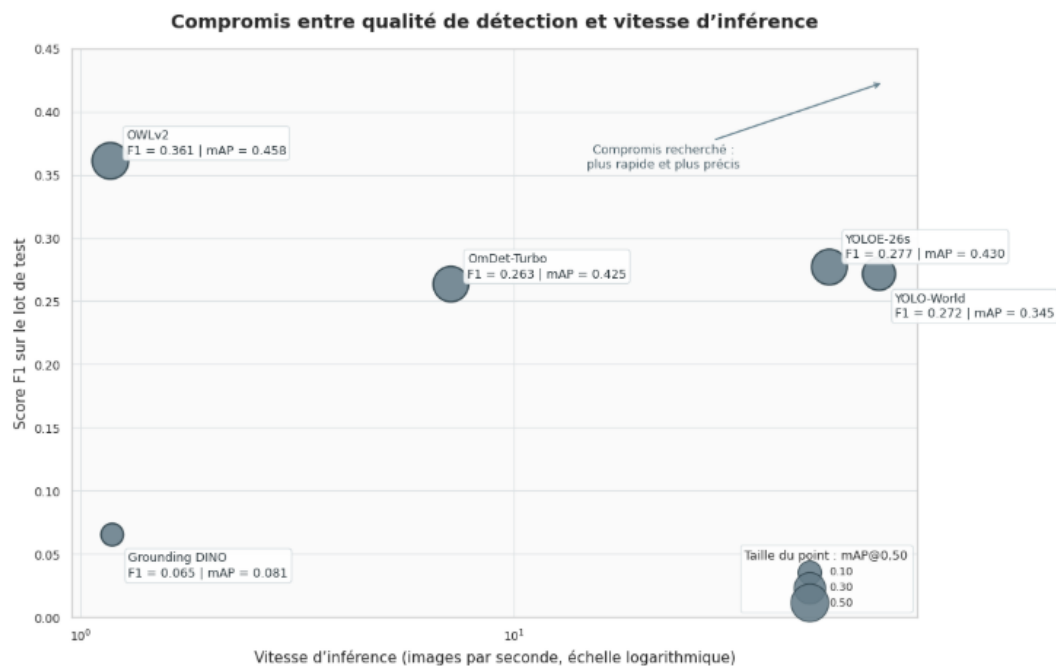
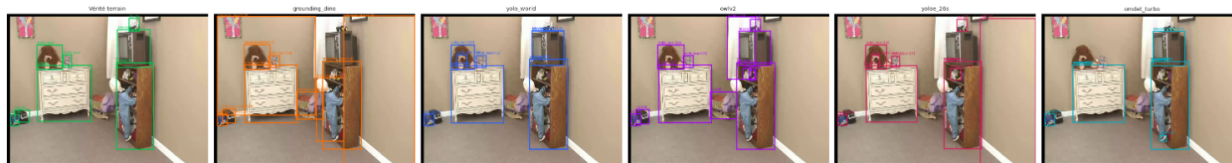


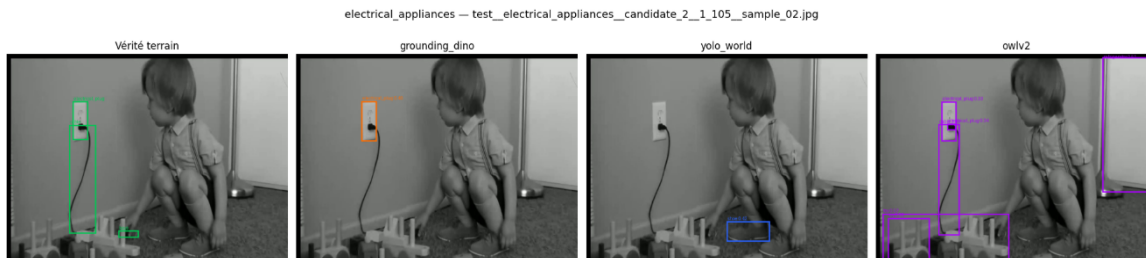
Figure 6: Compromis observé entre F1, mAP@0.50 et vitesse d'inférence sur le benchmark d'objets. La taille des points représente la mAP@0.50.

La figure 6 met en évidence le compromis entre qualité de détection et vitesse d'inférence.

YOLOE-26s est retenu : sa mAP@0.50 (0.430) reste proche du meilleur score (OWLv2, 0.458) tout en permettant un débit environ 40 fois supérieur, compatible avec un traitement vidéo. Son F1 de 0.277 reste toutefois modeste : YOLOE-26s constitue le meilleur compromis parmi les modèles testés, mais pas encore une solution de détection suffisamment fiable pour un produit final – une limite assumée et discutée en section 4.7.



(a) Scène contenant plusieurs meubles et petits objets.



(b) Cas de sécurité autour d'une prise électrique.

Figure 7: Comparaison qualitative entre la vérité terrain et les sorties des détecteurs. Ces exemples complètent les métriques agrégées en montrant les omissions, les boîtes imprécises et les différences de vocabulaire.

La figure 7 montre pourquoi le choix ne peut pas reposer uniquement sur la mAP. Dans la scène encombrée, les modèles ne retrouvent pas tous les mêmes instances et peuvent regrouper différemment des éléments proches. Dans le cas de la prise, certains détecteurs localisent l'objet dangereux tandis que d'autres l'omettent ou produisent des boîtes sur des éléments voisins. Une erreur de ce type est plus critique qu'une omission d'objet décoratif, car elle prive le module de décision d'une information de risque. Ces deux images restent toutefois des exemples : elles expliquent des modes d'échec, mais ne remplacent pas l'évaluation quantitative sur les 48 images de test. Les courbes de calibration et les résultats détaillés par classe sont conservés en annexe C, respectivement dans les figures 8 et 9.

Proximité et stabilisation temporelle. La distance entre les centres des boîtes enfant/objet est normalisée par la diagonale de l'image :

$$d_{\text{norm}} = \frac{\sqrt{(x_c - x_o)^2 + (y_c - y_o)^2}}{\sqrt{W^2 + H^2}}, \quad \text{proche si } d_{\text{norm}} \leq 0.20 \text{ (seuil empirique provisoire)}. \quad (2)$$

Un objet n'est conservé, après fusion sur les 16 *frames* d'une vidéo, que s'il est détecté sur au moins deux *frames*, ou avec une confiance maximale ≥ 0.60 (0.35 pour les objets à haut risque – couteau, prise électrique, médicament, verre) : ce seuil asymétrique privilégie délibérément le rappel des objets dangereux au prix de quelques fausses détections tolérées, qui pourront être filtrées grâce au contexte temporel. Aucune détection n'est interpolée entre *frames* : une absence de détection reste une absence, jamais une supposition.

Mesure	Valeur
Détections d'objets	749
Distances calculables (enfant détecté)	640 (85.45 %)
Distances non calculables	109 (14.55 %)
Catégories effectivement détectées	31/43
Latence moyenne YOLOE-26s	20.58 ms/image (~48.6 img/s)

Tableau 9: Bilan de la fusion géométrique sur le lot d'intégration (12 vidéos, 749 détections d'objets).

Format de sortie. Les sorties de VideoMAE (Module 2), YOLO26m et YOLOE-26s sont fusionnées dans un JSON unique par vidéo :

```
{ "video_id": "...",
  "activity": {"predicted_class": "objects_falling",
               "confidence": 0.82, "is_abnormal": true,
               "confidence_level": "high_confidence"},
  "child_detection": {"detected": true, "detection_rate": 0.9375},
```

```
"objects_sorted_by_distance": [
  {
    "name": "book", "confidence": 0.78,
    "distance_normalized": 0.12, "presence_ratio": 0.50
  }
]
```

Cet extrait illustre le schéma d'échange ; les valeurs numériques sont présentées à titre d'exemple de structure. Dans le pipeline, chaque document est également validé avant d'être transmis au module de décision.

4.5 Modules 4 et 5 – Décision du risque et notification

Objectif. Le Module 4 reçoit uniquement le JSON produit par les modules de perception (jamais la vidéo brute), afin que sa décision reste explicable à partir d'informations traçables, et doit produire une décision binaire $y \in \{\text{hazard}, \text{no_hazard}\}$. Une activité anormale n'est en effet pas systématiquement dangereuse (courir ou sauter peuvent être sans risque), ce qui justifie ce module de raisonnement contextuel plutôt qu'une règle directe activité $\Rightarrow$ danger.

Protocole. 48 vidéos ont été sélectionnées (2 vidéos $\times$ 12 classes $\times$ 2 ensembles), séparées en 24 vidéos de calibration et 24 de test, avec une vérité terrain **hazard/no_hazard** annotée manuellement – distincte du label d'activité, car le rôle du module est précisément d'ajouter une interprétation contextuelle. Six configurations finales ont été comparées sur des entrées identiques : une baseline déterministe, quatre LLM open-source (Qwen2.5-3B, TinyLlama-1.1B, Mistral-7B-Instruct et SmolLM2-1.7B), chacun dans le meilleur mode de *prompting* sélectionné sur calibration, puis l'adaptation QLoRA du meilleur modèle de base [12].

La vérité terrain a été établie par une seule annotatrice et la catégorie d'activité restait visible pendant l'annotation. Les labels de danger ont bien été définis séparément des labels d'activité, mais l'absence de second annotateur et d'accord inter-annotateurs constitue une source possible de biais. Le test contient ainsi 16 situations annotées **hazard** et 8 **no_hazard**.

Paramètre	Valeur
Seuil de confiance de l'activité	0.20
Seuil de distance entre boîtes	0.02
Seuil de distance entre centres	0.10
Ratio minimal de présence temporelle	0.125

Tableau 10: Baseline par règles : seuils calibrés (jeu de calibration).

Méthode	Mode	Acc.	Préc.	Rappel	F1	FP	FN	JSON valide
Règles déterministes	Frozen	0.708	0.800	0.750	0.774	3	4	100 %
Qwen2.5-3B	Few-shot	0.542	1.000	0.313	0.476	0	11	100 %
TinyLlama-1.1B	Zero-shot	0.750	0.813	0.813	0.813	3	3	0 %
Mistral-7B	Few-shot	0.792	0.923	0.750	0.828	1	4	100 %
SmolLM2-1.7B	Few-shot	0.333	0.000	0.000	0.000	0	16	100 %
QLoRA-Mistral	Fine-tuned	0.625	0.706	0.750	0.727	5	4	100 %

Tableau 11: Comparaison finale des méthodes de décision sur le test (24 vidéos, 16 hazard / 8 no_hazard).

Lecture des résultats. Un taux de JSON valide $\geq 95\%$ a été posé comme condition d'éligibilité opérationnelle avant toute comparaison de qualité de décision. TinyLlama, malgré le meilleur rappel brut, est ainsi disqualifié : aucune de ses 24 sorties n'est un JSON exploitable automatiquement (ses métriques proviennent d'un parsing *a posteriori* non fiable). Qwen2.5-3B et SmolLM2-1.7B, à l'inverse, sont trop prudents : une précision parfaite ne compense pas 11 à 16 dangers manqués sur 16. **Mistral-7B-Instruct en few-shot** [11] offre le meilleur compromis opérationnel : F1 le plus élevé (0.828) parmi les méthodes exploitables, un seul faux positif, et 100 % de JSON valides – au prix d'une latence plus élevée (13.3 s/vidéo).

Adaptation QLoRA – résultat négatif documenté. Mistral a été adapté par QLoRA (quantification 4 bits NF4, $r = 8$, $\alpha = 16$, dropout 0.05, taux d'apprentissage 2×10^{-5} , 20 971 520 paramètres entraînaibles soit 0.2885 % du modèle, 24 exemples de calibration et 2 époques, supervision limitée aux tokens de réponse). Le

résultat sur le test est *moins bon* que le few-shot non adapté : F1 0.828 $\rightarrow$ 0.727, faux positifs 1 $\rightarrow$ 5, à rappel identique (0.750). L'analyse qualitative montre que le modèle adapté sur-pondère la proximité d'un objet et sous-pondère la dangerosité intrinsèque de certaines activités (2 des 4 erreurs concernent `head_stuck`, où le modèle exige à tort un objet proche pour déclarer un danger). Avec seulement 24 exemples supervisés, ce résultat négatif ne condamne pas QLoRA en général mais montre que l'adaptation n'apporte pas de gain avec un jeu aussi réduit. **La configuration finale retenue est donc Mistral-7B + few-shot**, sans adaptation.

Sortie du système (Module 5). Le prototype journalise les décisions et leurs justifications dans la console et dans des fichiers JSON/CSV. Le format de sortie effectivement prévu est de la forme suivante :

```
{
  "video_id": "...",
  "hazard": "hazard",
  "reason": "Child is interacting with a sharp object."
}
```

Cette preuve de concept valide le contrat de sortie sans prétendre constituer une application de notification déployée. Une base de données persistante, une interface graphique et l'envoi temps réel d'alertes n'ont pas été implémentés, faute de temps. Le choix a été de concentrer l'effort sur la qualité des données, la comparaison expérimentale des modèles et la robustesse des interfaces entre modules : une application en aval n'aurait pas compensé les erreurs d'un pipeline de perception insuffisamment étudié. Le stockage et la notification temps réel relèvent donc des perspectives, et non des contributions revendiquées.

4.6 Tests logiciels

Chaque module a fait l'objet de contrôles à plusieurs niveaux, condition nécessaire selon les consignes du stage (« un développement sans test est un développement mort-né »).

Niveau	Nombre	Exemples de vérifications
Unitaires	5	Conversions numériques et boîtes, cas de proximité <i>near/far</i>
Module	10	12 vidéos, 192 frames, confiances bornées, 12 sorties VideoMAE, distances normalisées
Intégration	4	Cohérence des identifiants, 12 JSON, 16 frames/JSON, conservation des objets
Système	5	Sorties définitives, modèles documentés, JSON final relisible sur disque
Total	24	Contrôles exécutables automatiquement dans le notebook

Tableau 12: Contrôles logiciels définis dans le notebook final du Module 3.

Pour le Module 4, les tests unitaires couvrent notamment le *parseur* de décision (une sortie non conforme ne doit jamais être silencieusement interprétée comme `no_hazard` – une erreur de format ne doit jamais être assimilée à une absence de danger), les tests d'intégration vérifient la cohérence des identifiants vidéo entre modules. Ces vérifications internes ne remplacent toutefois pas l'exécution finale du notebook de bout en bout, distinguée dans la section suivante.

4.7 Validation de bout en bout et limites

Un notebook dédié a été préparé pour vérifier la chaîne complète vidéo $\rightarrow$ YOLO26m $\rightarrow$ VideoMAE $\rightarrow$ YOLOE $\rightarrow$ distances $\rightarrow$ JSON $\rightarrow$ Mistral. Les cas retenus proviennent du test du Module 4 : ils ne forment donc pas un nouvel ensemble destiné à estimer la généralisation, mais un test fonctionnel des interfaces. Au moment de la consolidation de ce rapport, la dernière version du notebook n'avait pas encore produit une sortie archivée ; aucun taux de réussite de bout en bout n'est donc avancé. Cette absence est traitée comme une limite de validation, et non remplacée par les résultats d'une ancienne version du pipeline.

Rang	Limite	Piste d'amélioration
1	Détection enfant insuffisante sur le lot d'intégration	Enrichir les cas d'occlusion et de flou
2	Détection ouverte : F1 de 0.277	Étendre le benchmark et adapter YOLOE
3	Scores VideoMAE non calibrés	Calibrer les probabilités
4	Absence de suivi d'identité	Ajouter ByteTrack ou BoT-SORT
5	Distance limitée au plan 2D	Estimer la profondeur
6	Validation E2E finale non archivée	Exécuter et archiver le notebook gelé

Tableau 13: Principales limites identifiées et priorités d'amélioration.

Les interfaces entre modules ont été intégrées progressivement et leurs sorties ont été contrôlées séparément ou par groupes de composants. En revanche, faute de sortie archivée pour une exécution complète, la chaîne de bout en bout ne peut pas être considérée comme validée. Le résultat est donc un prototype de recherche modulaire et testable, non un dispositif de sécurité prêt au déploiement. Les performances, notamment le F1 de 0.828 du Module 4, doivent être lues comme une validation expérimentale à petite échelle : avec 24 vidéos de test, une seule erreur modifie l'exactitude de 4.17 points.

5 Méthode de gestion de projet

5.1 Organisation adoptée

Le stage a été mené principalement en autonomie, sans appliquer formellement un cadre Scrum. Il a suivi une démarche expérimentale itérative : comprendre le problème et les travaux antérieurs, rechercher des modèles récents adaptés à chaque module, établir un protocole d'évaluation commun, entraîner et comparer les modèles, analyser les erreurs, corriger les biais du protocole, puis intégrer progressivement les modules. Les propositions d'expérience étaient discutées avec l'encadrant avant leur exécution, et les résultats intermédiaires présentés après chaque étape importante – un fonctionnement souple, adapté à un travail exploratoire, mais qui imposait de documenter précisément chaque configuration (paramètres, graine aléatoire, checkpoints) pour pouvoir comparer les essais *a posteriori*. Les orientations principales étaient discutées avec le professeur Muhammad Khan ; les questions techniques pouvaient être examinées plus directement avec Shehzad Ali au laboratoire. Les échanges étaient réguliers, sans réunion quotidienne formalisée.

Le stage s'est déroulé du 14 avril au 14 août 2026. Les phases présentées dans le tableau 14 donnent leur ordre dominant ; elles se sont partiellement chevauchées en raison des réentraînements et des retours sur le protocole expérimental.

Ordre	Phase	Livrable ou décision associée
1	Prise en main	Littérature, données, environnements et métriques
2	Perception	Comparaisons YOLO/RT-DETR puis VideoMAE/UniFormerV2/VideoMamba
3	Contexte	Détecteurs ouverts, distances et contrat JSON
4	Raisonnement	Baseline, quatre LLM, QLoRA et sélection de Mistral
5	Intégration	Contrôles logiciels, analyse des limites et préparation E2E

Tableau 14: Planning synthétique des tâches réalisées pendant le stage.

Difficulté	Réponse apportée
Domaine hors spécialité initiale	Apprentissage accéléré de PyTorch, de l'entraînement de réseaux profonds et des métriques de vision
Fiabilité de l'évaluation	Détection et correction de la fuite de données de RHAD (section 4.3) ; réentraînement des trois modèles candidats
Extraction du contexte	Arbitrage précision / robustesse / rapidité entre 5 détecteurs <i>open-vocabulary</i>

Tableau 15: Principales difficultés rencontrées.

5.2 Rétrospective : et si c'était à refaire ?

Deux choix seraient anticipés plus tôt. D'une part, le regroupement des clips par vidéo source (Module 2) aurait dû être vérifié dès la découverte du dataset, avant tout entraînement, plutôt qu'après avoir observé des performances suspectes : cela aurait évité de réentraîner l'ensemble des modèles candidats. D'autre part, un petit ensemble de cas de test pour le Module 4 aurait pu être annoté en parallèle des modules de vision plutôt

qu'après coup, ce qui aurait permis d'itérer plus tôt sur le format du *prompt*. Ces ajustements n'ont pas remis en cause la faisabilité du projet, mais auraient réduit le temps passé à revenir sur des étapes déjà considérées comme validées.

6 Bilan et conclusion

6.1 Bilan : ce que ce stage m'a apporté

Ce stage a permis de construire les principales briques d'un système intelligent de surveillance des dangers pour les enfants, organisé en cinq modules articulés autour d'une interface JSON explicite. La comparaison de neuf configurations de détection ont conduit à sélectionner YOLO26m pour localiser l'enfant ; trois modèles de reconnaissance vidéo ont été évalués sur un découpage de RHAD corrigé après identification d'une fuite de données – une contribution méthodologique à part entière. VideoMAE, retenu, détecte l'intégralité des événements anormaux du test au niveau binaire mais échoue systématiquement sur la classe `objects_falling`, ce qui a motivé une expérience d'enrichissement synthétique dont le résultat négatif a été documenté plutôt qu'occulté. Un pipeline d'extraction du contexte a ensuite été développé, comblant le manque identifié dans l'état de l'art, puis six configurations de décision du risque ont été comparées (jeux de calibration/test disjoints, condition d'éligibilité sur la validité du format JSON), aboutissant à la sélection de Mistral-7B en few-shot.

Au-delà des résultats obtenus, ce stage a constitué une première expérience concrète du travail de recherche en intelligence artificielle : importance de l'audit des données, de la reproductibilité des expériences (seed fixée, séparation stricte calibration/test à chaque étape) et de l'analyse critique des métriques au-delà d'un score global. Ces acquis complètent ma formation d'ingénieure en informatique par une pratique concrète de l'expérimentation et de la conception de systèmes modulaires d'intelligence artificielle.

6.2 Conclusion

Les principales limites de ce prototype (détection de l'enfant en cas d'occlusion, F1 modeste du détecteur d'objets, absence de profondeur et de suivi temporel, taille réduite des jeux d'évaluation) sont détaillées et priorisées dans le tableau 13. Les perspectives directes sont la diversification des situations normales du jeu de test, la génération de données synthétiques sans filigrane, la calibration des confiances des modèles, le profilage puis l'optimisation du temps d'inférence de bout en bout, et une évaluation auprès d'utilisateurs réels, préalable indispensable à tout déploiement.

Références

- [1] S. Ali, M. T. Islam, I. H. Lee, M. Xiong, M.-S. Dao, S. Anwar, S. Bakshi et K. Muhammad, *Towards Hazardous Activity Recognition for a Novel Real-World Dataset*, Proceedings of the 33rd ACM International Conference on Multimedia, 2025, doi: 10.1145/3746027.3754795.
- [2] Z. Tong, Y. Song, J. Wang et L. Wang, *VideoMAE: Masked Autoencoders are Data-Efficient Learners for Self-Supervised Video Pre-Training*, Advances in Neural Information Processing Systems, 2022.
- [3] K. Li et al., *UniFormerV2: Spatiotemporal Learning by Arming Image ViTs with Video UniFormer*, Proceedings of the IEEE/CVF International Conference on Computer Vision, 2023.
- [4] K. Li et al., *VideoMamba: State Space Model for Efficient Video Understanding*, European Conference on Computer Vision, 2024.
- [5] Y. Zhao et al., *DETRs Beat YOLOs on Real-Time Object Detection*, Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2024.
- [6] G. Jocher, J. Qiu, M. Liu, S. Lyu, F. C. Akyon et M. E. Kalfaoglu, *Ultralytics YOLO26: Unified Real-Time End-to-End Vision Models*, arXiv:2606.03748, 2026.
- [7] T. Cheng, L. Song, Y. Ge, W. Liu, X. Wang et Y. Shan, *YOLO-World: Real-Time Open-Vocabulary Object Detection*, Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2024.
- [8] S. Liu et al., *Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection*, European Conference on Computer Vision, 2024, arXiv:2303.05499.
- [9] M. Minderer, A. Gritsenko et N. Houlsby, *Scaling Open-Vocabulary Object Detection*, arXiv:2306.09683, 2023.

- [10] A. Wang, L. Liu, H. Chen, Z. Lin, J. Han et G. Ding, *YOLOE: Real-Time Seeing Anything*, Proceedings of the IEEE/CVF International Conference on Computer Vision, 2025, arXiv:2503.07465.
- [11] A. Q. Jiang et al., *Mistral 7B*, arXiv:2310.06825, 2023.
- [12] T. Dettmers, A. Pagnoni, A. Holtzman et L. Zettlemoyer, *QLoRA: Efficient Finetuning of Quantized LLMs*, Advances in Neural Information Processing Systems, 2023.
- [13] Sungkyunkwan University, *History and Overview*, https://www.skku.edu/eng/About/s620/sub02_01.do, consulté en août 2026.

ANNEXES

A Paramètres principaux des expériences

Paramètre	Valeur
Frames de contexte par vidéo	16
Seuil enfant principal / de secours	0.25 / 0.10
Seuil YOLOE-26s (calibré)	0.14
NMS IoU YOLOE	0.50
Seuil provisoire de proximité	0.20
Seuil d'activité AT	0.354459

Tableau 16: Paramètres principaux du Module 3.

B Conventions d'annotation manuelle (CVAT, benchmark objets)

Extrait des règles appliquées lors de la construction de la vérité terrain du benchmark de détection d'objets (section 4.4) : boîte serrée autour de la partie visible ; une instance physique = une boîte ; enfants annotés (child), adultes non annotés ; objet flou ou ambigu non annoté (pour éviter une fausse vérité terrain) ; éléments appartenant à un même objet regroupés (ex. porte d'un placard) ; structures répétitives homogènes regroupées en une seule instance (ex. barreaux) ; chevauchement de boîtes autorisé (ex. enfant/chaise) ; surfaces de fond (sol, plafond) non annotées sauf participation directe à une activité.

C Résultats complémentaires du benchmark d'objets

	model	selected_threshold	tp	fp	fn	precision	recall	f1	fp_per_frame	frames_without_detection
0	grounding_dino	0.33	6	54	119	0.1000	0.048	0.0649	1.1250	23
1	yolo_world	0.25	19	23	106	0.4524	0.152	0.2275	0.4792	31
2	owlv2	0.33	42	31	83	0.5753	0.336	0.4242	0.6458	18

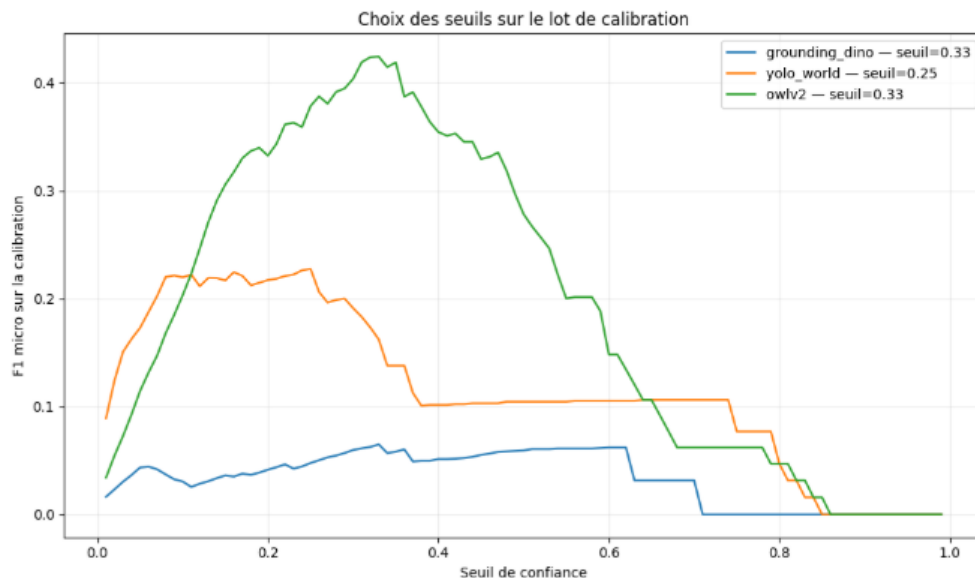


Figure 8: Choix des seuils de confiance sur le lot de calibration lors du benchmark initial à trois détecteurs.

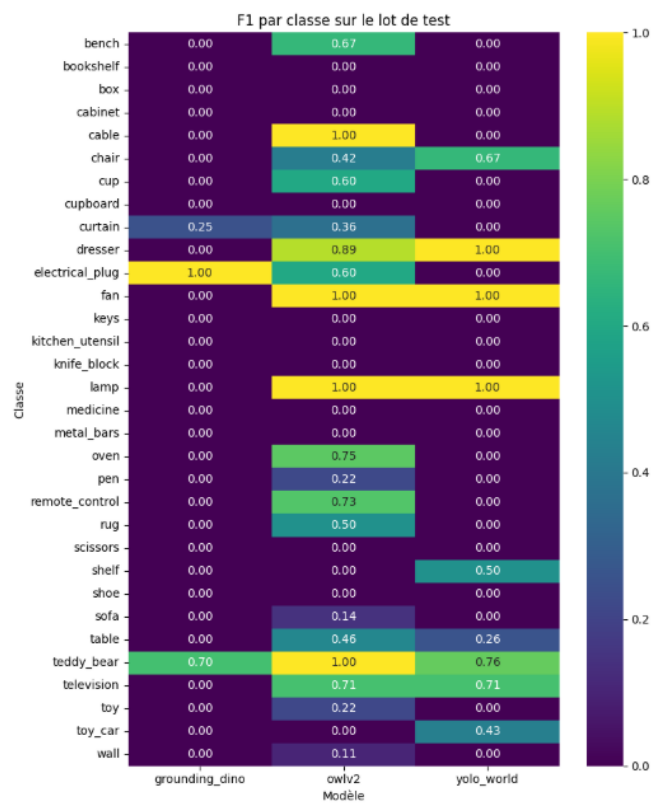


Figure 9: F1 par classe sur le lot de test du benchmark initial. La forte hétérogénéité confirme que le score agrégé ne suffit pas à caractériser les erreurs.